

Embedded Programming

Field-Programmable Gate Array (FPGA)

Dr. Angela Cratere

Department of Advanced Computing Sciences

Maastricht University

2025-2026

angela.cratere@maastrichtuniversity.nl



Course outline – Embedded Programming

- Primer on computer architecture
- The STM32U5 and ARM Cortex-M33 architectures
- Data representation and bitwise operations
- Pointers and pointer arithmetic using C
- ARM ISA and simple assembly language
- Fun with GPIOs and clocks
- **Week 5: Introduction to the FPGA technology and Vitis AI**

Learning objectives (week 5)

- Demonstrate an understanding of **FPGA** architecture and its main building blocks
- Demonstrate an understanding of the role of **Deep Learning Processing Units (DPUs)** as dedicated hardware accelerators for **neural network inference** on FPGA-based platforms
- Understand the high-level architecture of a DPU and its interaction with the programmable logic and processing system
- Gain an understanding of the **Vitis AI development environment** and toolchain (model preparation, quantization, and compilation)

Reading material (week 5)

Books:

- L. Crockett, D. Northcote, C. Ramsay, F. Robinson, and B. Stewart - **Exploring Zynq MPSoC with PYNQ and Machine Learning Applications**
- Dirk Koch, **Partial Reconfiguration on FPGA: Architectures, Tools and Applications**, Springer, 2013

Different datasheets for reference during the lectures and tutorials:

- Vitis AI User Guide (UG1414)
- DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)

Useful papers:

- Gholami *et al.*, 2021, *A Survey of Quantization Methods for Efficient Neural Network Inference*

General Purpose vs. Special Purpose System

General Purpose



CPU

General Purpose
fixed architecture

Flexible at Sw lvl

Red. performance

Easy
development

Power
consumption



GPU

Fixed specialized
architecture

Flexible at SW lvl

High performance

Mid-lvl ease of
development

Power
consumption



FPGA

Reconfigurable
architecture

Flexible at HW/SW

High performance

Not so easy
development

Power
consumption



ASIC

(Application-specific integrated circuit)

Fixed specific
architecture

Non flexible

High Performance

Complex
development

Power
consumption

Special Purpose

Flexible: Can execute any program

Easy to program & use

Not the best performance & efficiency

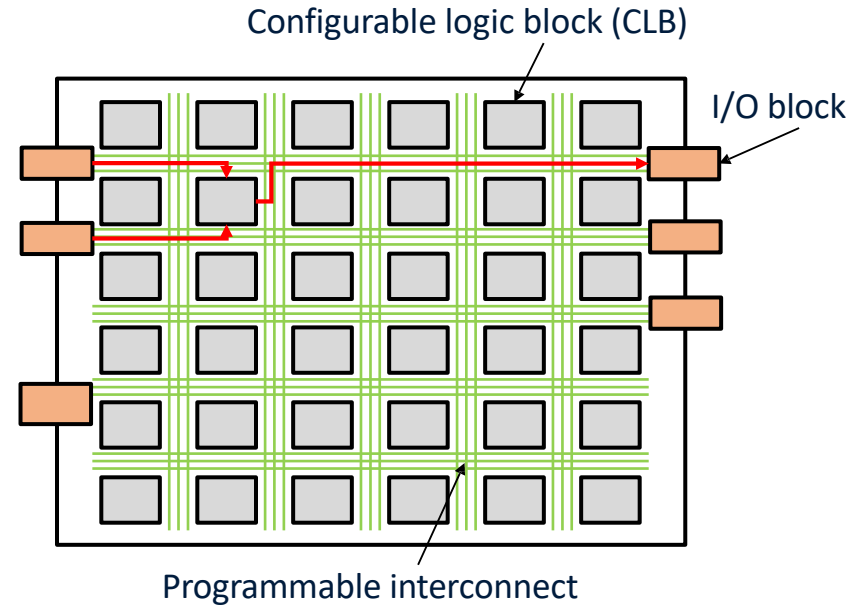
Efficient & High performance

(Usually) Difficult to program & use

Inflexible: Limited set of programs

FPGA – Field Programmable Gate Array

- An **integrated circuit** that can be **configured** (hardware-programmed) by the user even after manufacturing to emulate any digital circuit (as long as there are enough resources)
- An FPGA is an array of **Configurable Logic Blocks** (CLBs). A CLB is a **programmable basic logic cell** that can implement both **combinational** and **sequential** logic
- A matrix of **programmable interconnect** surrounds the CLBs
- Programmable I/O cells surround the core



FPGA Programmability

Three different levels of programmability:

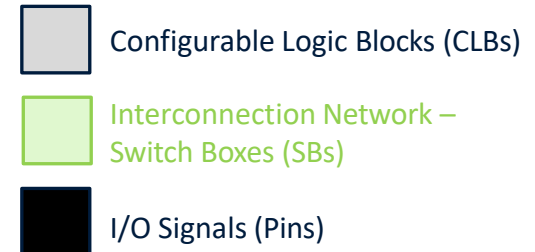
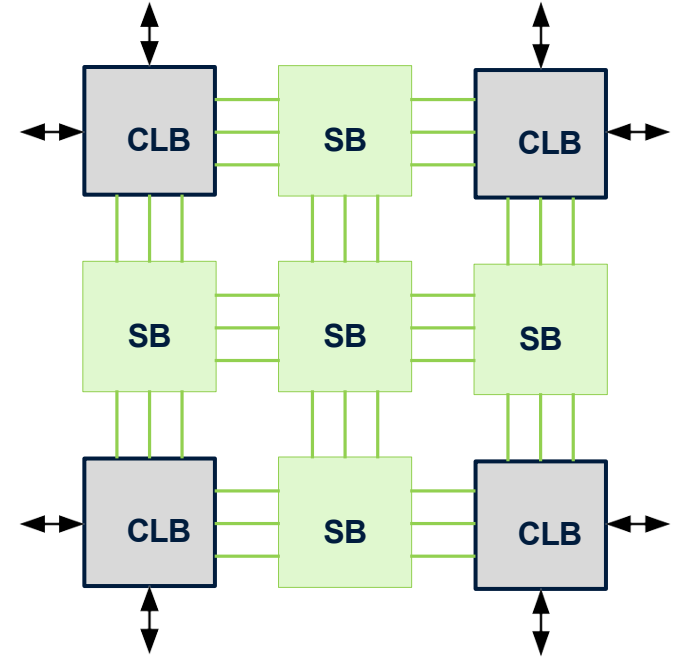
- **Logic programmability of CLBs**
- **Interconnect programmability**
- **I/O programmability**

FPGAs fill the gap between *software* and *hardware*

- Achieves higher performance than software
- Maintains more flexibility than fixed hardware

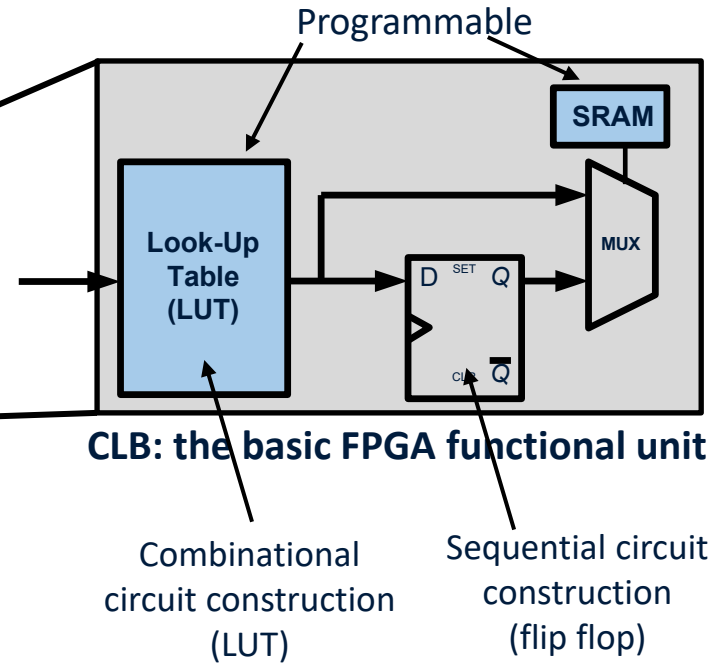
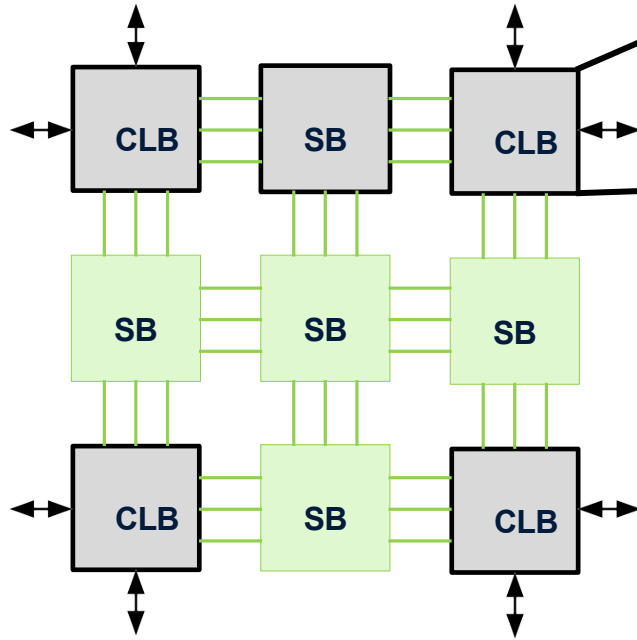
Very regular design
allowing for high-
density chips

Can be
complemented by
specialized blocks
(e.g., RAM or DSP)



CLB Structure

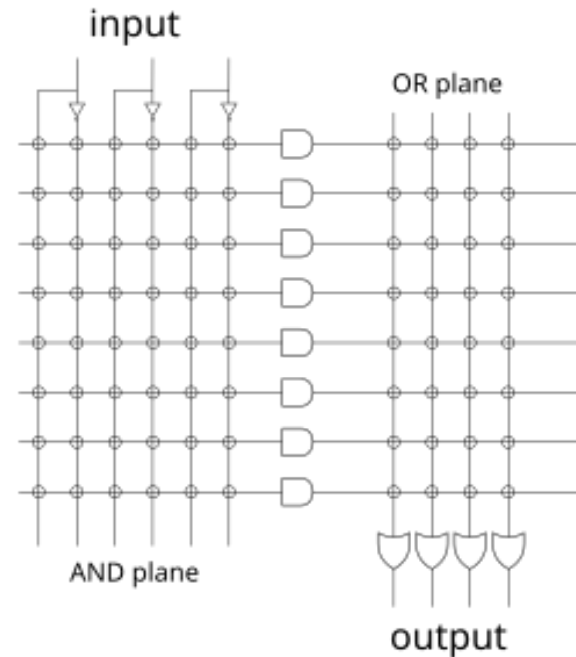
Both **combinational** and **sequential** logic



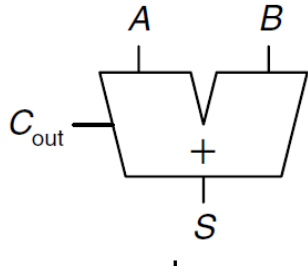
-  Configurable Logic Blocks (CLBs)
-  Interconnection Network – Switch Boxes (SBs)
-  I/O Signals (Pins)

Programmable Logic Array (PLA)

- FPGAs are a more flexible and sophisticated evolution of **PLAs**, which emerged in the 1980s as some of the earliest *Programmable Logic Devices* (PLDs).
- A PLA device consists of a **programmable array of AND gates** connected to a **programmable array of OR gates** allowing the implementation of any **combinational** logic functions (typically expressed in sum-of-products)
- The programmable part is implemented using fusible links or programmable read-only memories (PROMs), which store the configuration bits that define the interconnections in the AND/OR arrays (configuration memories). Hardware description languages (HDLs).

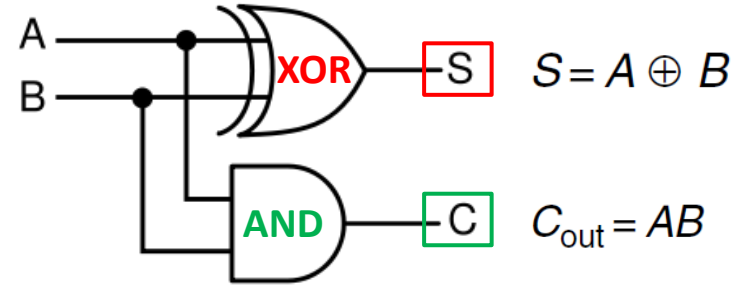


Half hadder with a PLA device



Truth table

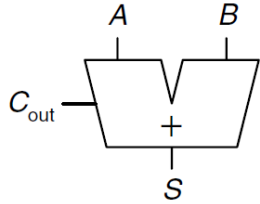
A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



How can it be implemented in a PLA?

- Write the Boolean equations in sum-of-products of the inputs (minterms)
- Configure the AND plane to generate the required product terms
- Configure the OR plane to form the output sums

Half hadder with a PLA device



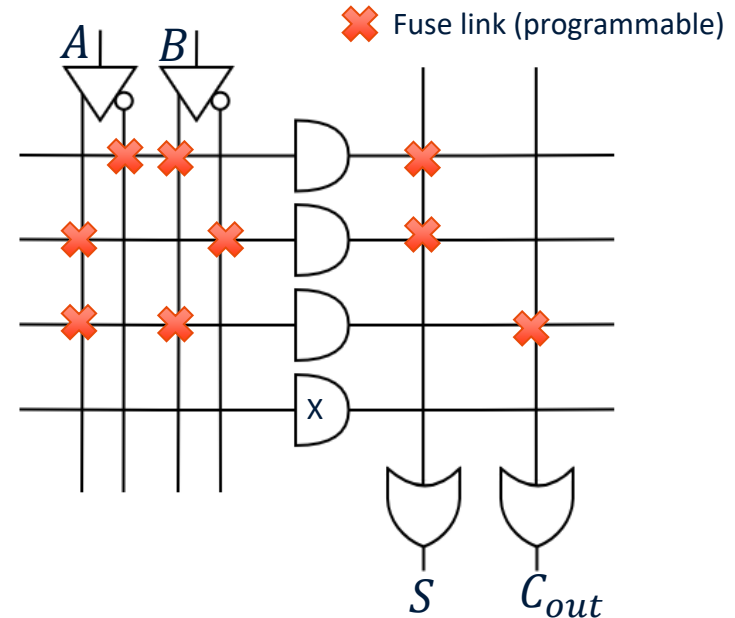
1. Write the Boolean equations in **sum-of-products** (minterms) form
This means expressing each output as an OR (sum) of AND terms (products) of the inputs and their complements, derived directly from the rows of the truth table where the output is equal to 1.

Truth table

Product terms	A	B	C_{out}	S
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	1	0	1
$A\bar{B}$	1	0	0	1
AB	1	1	1	0

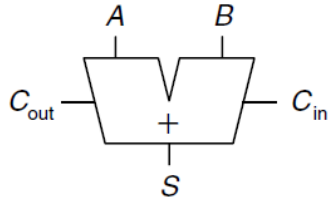
- $S = \bar{A}B + A\bar{B}$
- $C_{out} = AB$

2. Configure the AND (products) and OR (sum) matrices



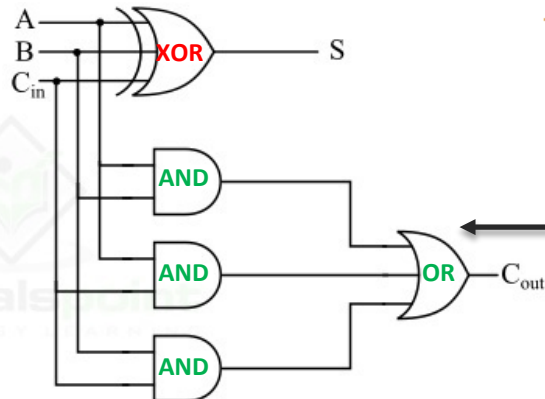
The advantage? Flexibility!

Full hadder with a PLA device



$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

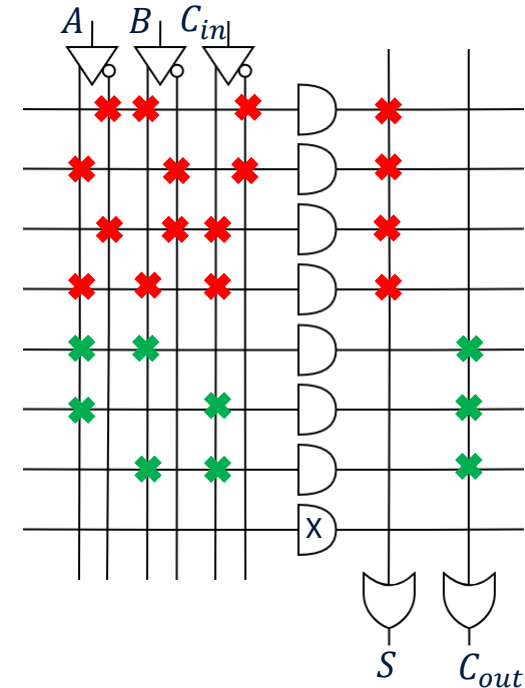
Product terms

$$\bar{A}\bar{B}\bar{C}_{in}$$

$$\bar{A}\bar{B}C_{in}$$

$$ABC_{in}$$

$$S = \bar{A}\bar{B}\bar{C}_{in} + \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in}$$



FPGAs are more advanced PLAs!

PLA:

- Consists in an array of AND and OR gates. The basic logic element is an AND/OR gate.
- Can implement any **combinational** logic function by programming the connections between the AND and OR planes.
- Has a fixed hardware structure with programmable connections

FPGA:

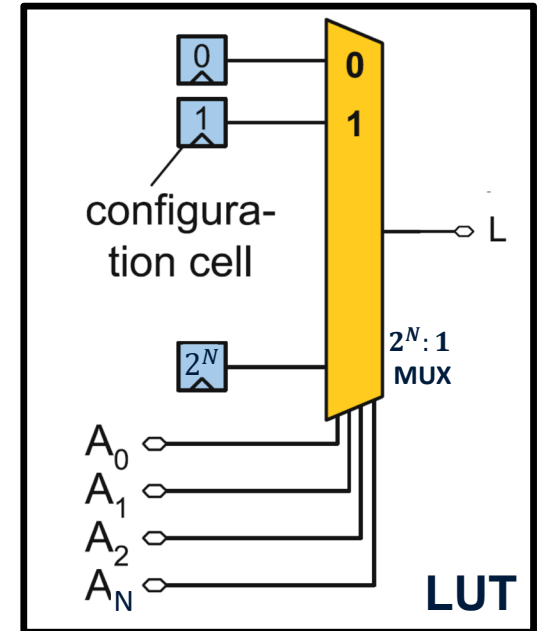
- Consists of an array **CLB**. Each CLB can implement both **combinational** and **sequential** logic.
 - The **combinational logic** is not implemented using explicit logic gates, but through **Look-Up Tables (LUTs)**
 - The **sequential logic** is implemented using **flip-flops (registers)** inside the CLBs
- In addition, the connections between different CLBs are also programmable through a network of programmable interconnects (switch boxes)

What is a Look-Up Table (LUT)?

- A LUT is a small **programmable memory block** used to implement **combinational** logic functions inside an FPGA
- It stores the **truth table** of a logic function rather than computing the function with gates

Structure:

- A LUT with **N inputs** contains **2^N memory cells** (typically SRAM cells)
- The inputs act as **address lines** used to select one stored value
- The selected value is the **output** of the LUT
- Internally, a **multiplexer** with N selection lines (bits) reads the addressed memory cell to select the output
- Any Boolean function of N variables can be implemented by programming the stored bits

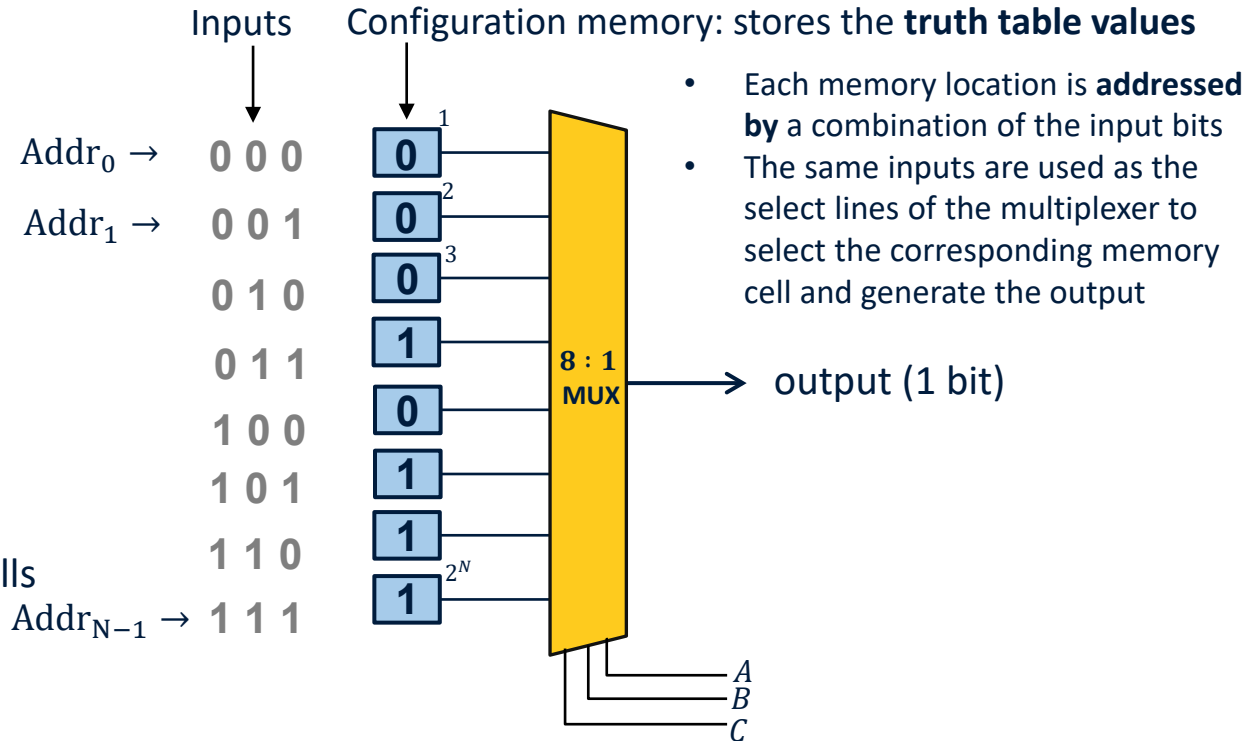


LUTs

- Let's implement a function of 3 inputs, using a 3-input LUT, that outputs '1' when there are more than one '1' in select inputs

A	B	C	Out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- $N = 3 \rightarrow 2^N = 8$ memory cells

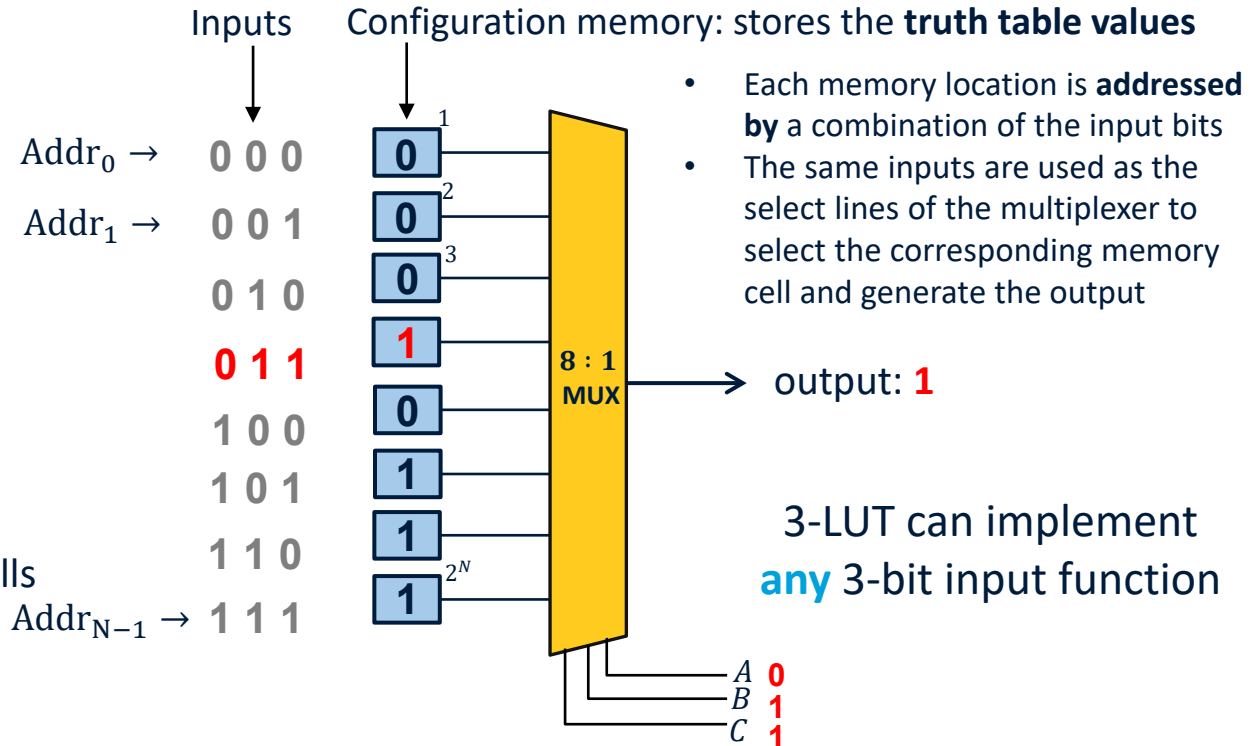


LUTs

- Let's implement a function of 3 inputs, using a 3-input LUT, that outputs '1' when there are more than one '1' in select inputs

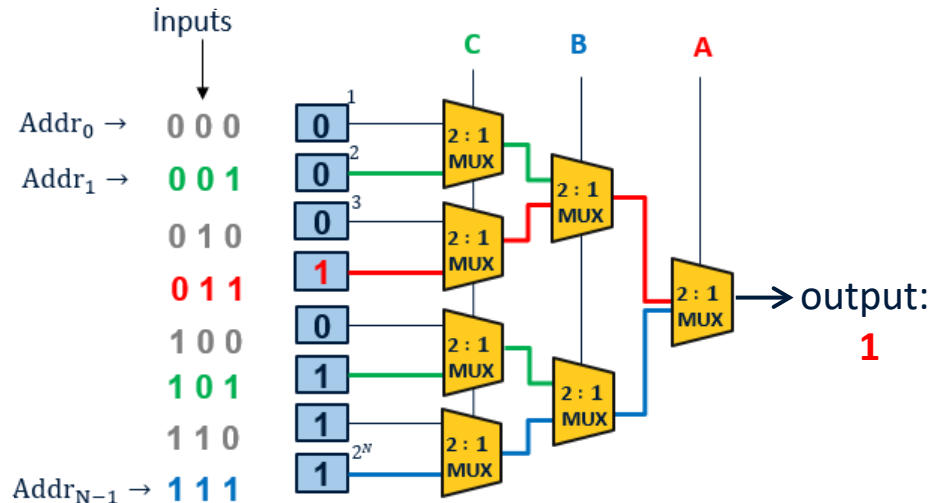
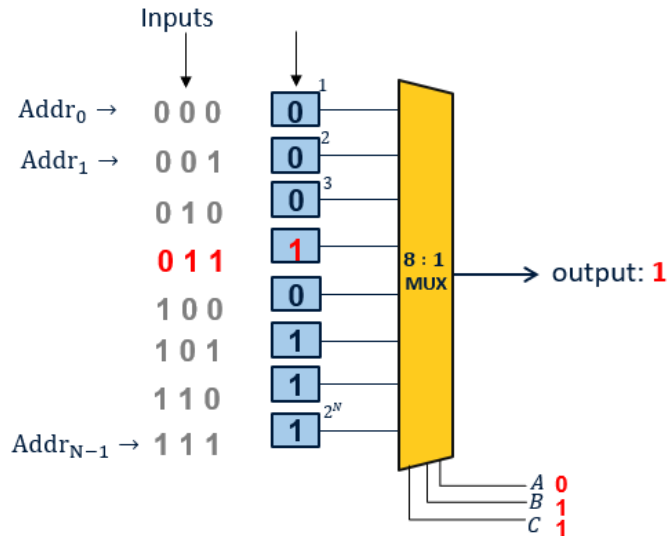
A	B	C	Out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- $N = 3 \rightarrow 2^N = 8$ memory cells

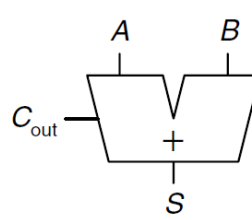


LUTs

- Actually, in real FPGA LUTs, large multiplexers are implemented as trees of 2:1 multiplexers because small multiplexers are faster, more area-efficient, and easier to implement in a regular FPGA architecture
- For example, a 3-input LUT (an 8:1 multiplexer) can be implemented using seven 2:1 multiplexers arranged in three stages, where each input bit acts as the select signal for one stage of the tree

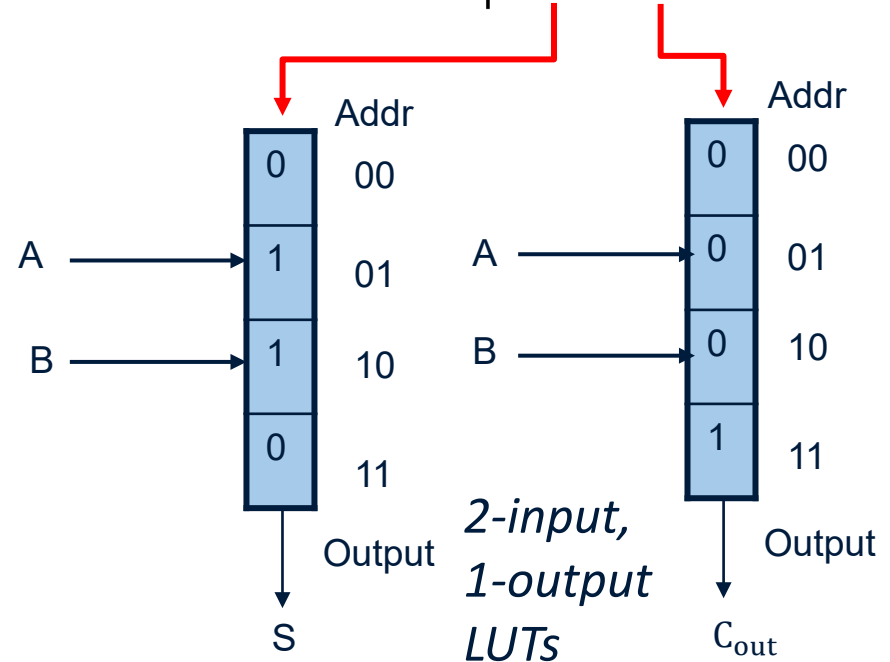


Half adder with a LUT



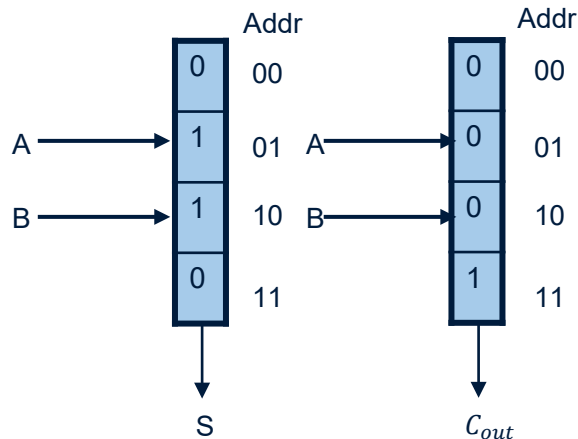
A	B	S	C _{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Two approaches to implement a half adder:
 - Using logical gates
 - Implementing the truth table in a LUT, storing the output inside a configuration memory**
- Since the half adder has two outputs, we can use two 2-input, 1-output LUTs
- A 2-input LUT contains $2^N = 4$ memory cells
- A and B act as address bits that select the appropriate stored output value

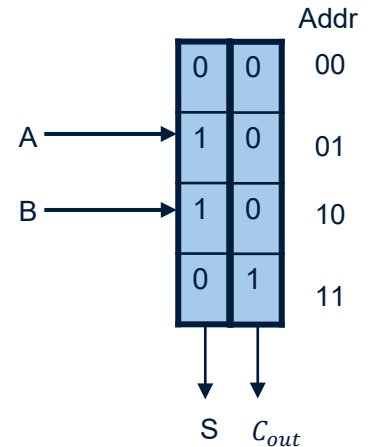


Half adder with a LUT

- Alternatively, we can use a single LUT with multiple outputs: **2-input, 2-output LUT**
- Since S and C_{out} functions depend on the same input signals A and B , the same address lines can be shared
- The configuration memory now stores **multiple bits** per address location, one for each output
- For each input combination, the LUT simultaneously produces both outputs by reading the corresponding memory word



2-input, 1-output LUTs



2-input, 2-output LUTs

Modern FPGA architectures – Xilinx

- Modern FPGAs typically use **6-input LUTs**, and combine *thousands* of them
- E.g., **Zynq UltraScale+ MPSoCs** can include $\approx$ **30K to 500K LUTs**. **Versal Adaptive SoCs** can scale up to **1 million LUTs**!

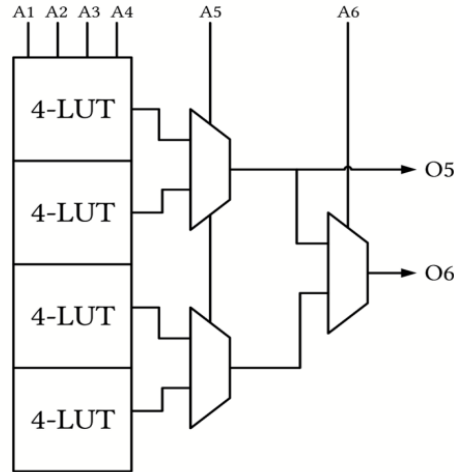
**Zynq
UltraScale+
MPSoCs**



**Versal
adaptive SoCs**

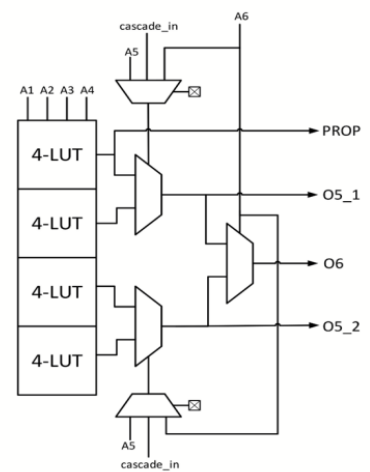


6-input, 2-output LUTs



(a) UltraScale 6LUT

6-input, 3-output LUTs



(b) Versal 6LUT

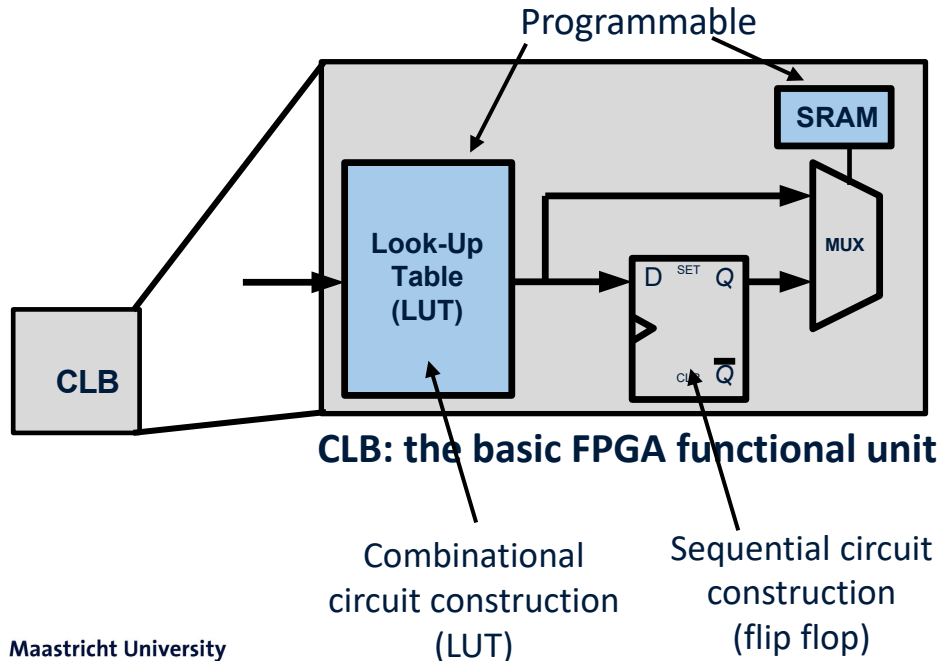
Gaide, B., et al. , (2019, February). Xilinx adaptive compute acceleration platform: Versal™ architecture.

Modern FPGA architectures – Xilinx

- The preferred approach is to use *small* LUTs combined together rather than one large LUT.
Why? **Scalability!**
- The size of a truth table grows exponentially with the number of inputs
- A LUT with i inputs and o outputs requires 2^i memory locations (SRAM cells) and $2^i \times o$ configuration bits
 - 6-input, 1-output LUT: $2^6 = 64$ memory locations $\rightarrow$ 64 configuration bits
 - 6-input, 2-output LUT: $2^6 = 64$ memory locations $\rightarrow$ 128 configuration bits
 - 64-input, 1-output LUT: $2^{64} \approx 1.84 \times 10^{19}$ memory locations and configuration bits!
Clearly, implementing large logic functions with a single LUT is not feasible.

Sequential Logic

- Combinational logic in an FPGA computes functions of the inputs, while flip-flops store the results over time, enabling sequential behavior such as registers, pipelines, and state machines

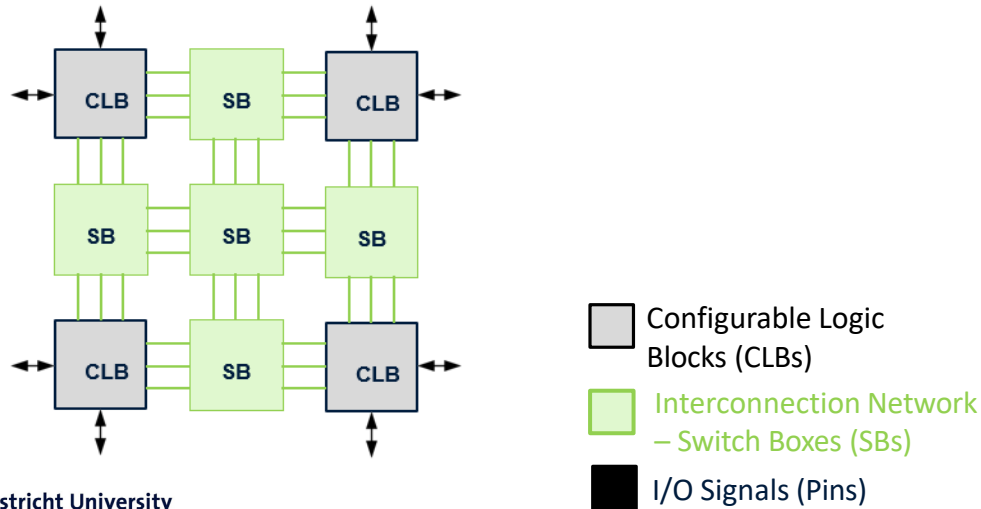


How to handle sequential logic?

- Add a **flip-flop** to the output of LUT
- Use a **multiplexer** to optionally bypass the flip-flop when only combinational logic is required, and then to **select** between the **combinational** LUT output and the **registered** (flip-flop) output
- Add a small programmable memory cell (SRAM) to **control the multiplexer select signal (configuration bit)**

Interconnection Network

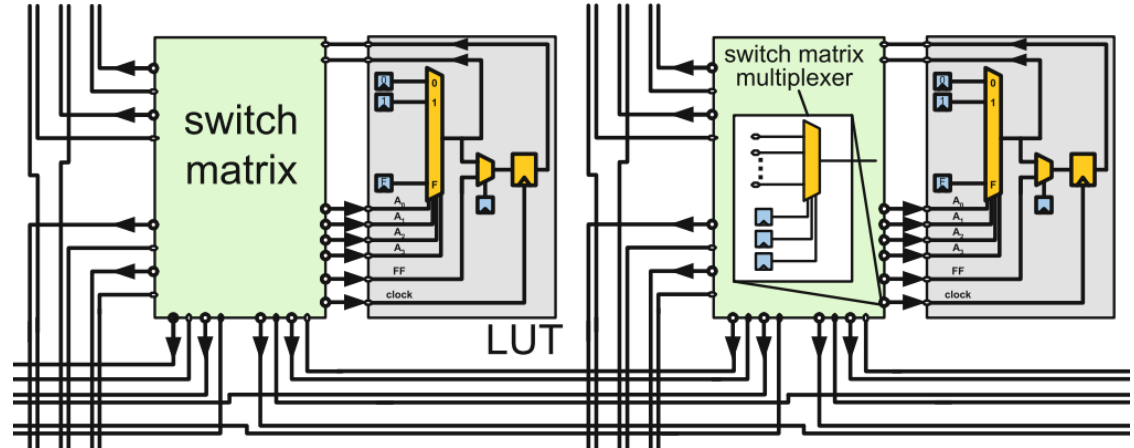
- With a reconfigurable LUT, we define the behavior of a digital circuit by loading truth tables into configuration SRAM cells
- However, a digital circuit is not only defined by logic functions, but also by **how logic elements are connected** together
- FPGAs also include **a reconfigurable interconnection network** that links CLBs



- Dense array of routing wires, which allow signals to travel from one CLB to another
- Programmable SBs, which enable connections between different routing wires and allow signals to change direction within the routing fabric

Interconnection Network

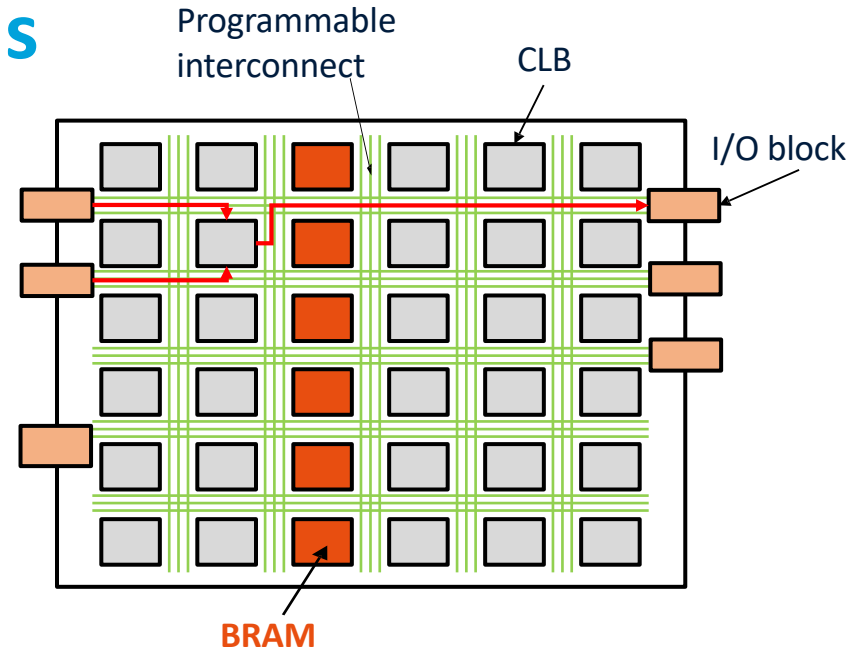
- These switchable interconnects (SBs) are again implemented with **multiplexers controlled by small configuration SRAM cells!**
- The configuration SRAM cells are connected to the select signals of the multiplexers
- The inputs of the multiplexer are the wires of the interconnection network



Dirk Koch, *Partial Reconfiguration on FPGA: Architectures, Tools and Applications*, Springer, 2013

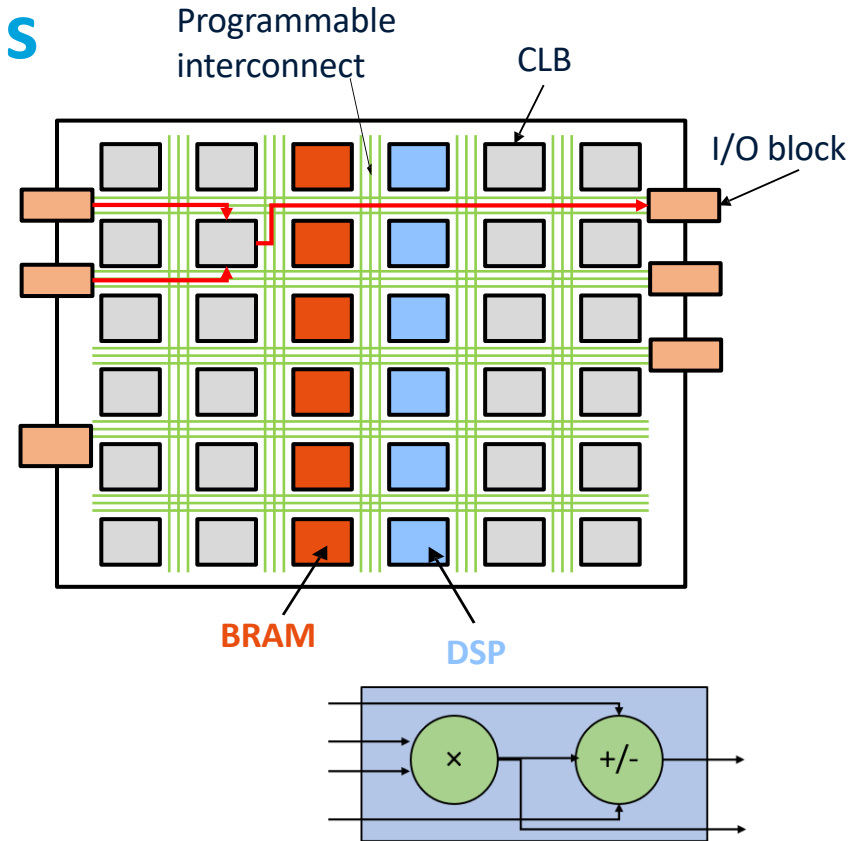
Modern FPGA architectures

- Modern FPGAs are not only composed of CLBs, but also include **specialized hardware blocks** (e.g., RAM, DSP) within the programmable fabric
- **Block RAM (BRAM).** CLBs contain small registers (flip-flops), storing ~ 1 bit/block) – tiny and inefficient for massively parallel memory access! For this, FPGAs include dedicated on-chip SRAM blocks. **Typical sizes: 18–36 Kbit per block, with several MB per chip available through multiple BRAM instances.**



Modern FPGA architectures

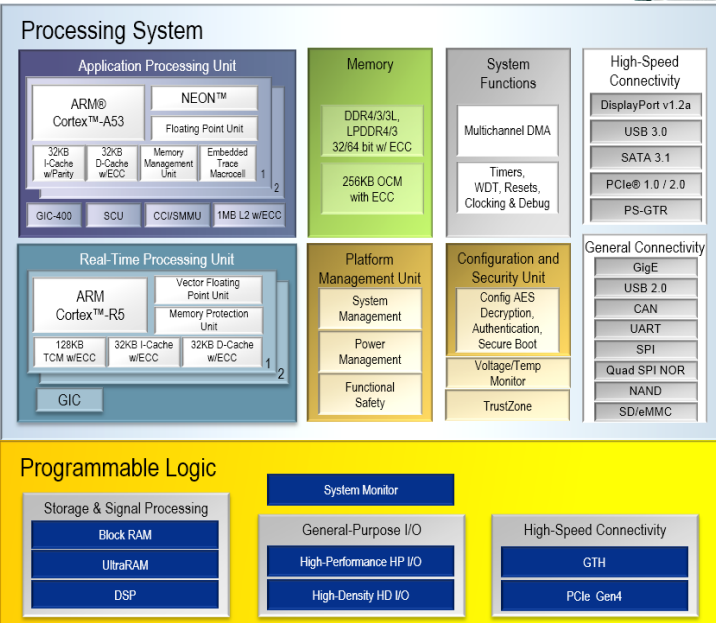
- Modern FPGAs are not only composed of CLBs, but also include specialized **hardware blocks**
- Digital Signal Processing (DSP) blocks.** While CLBs can implement arithmetic operations, doing so would require many LUTs and lead to inefficient designs. DSP blocks are **efficient ALU blocks** (each block provides an *adder* + *multiplier*) which provide higher performance and lower power consumption compared to LUT-based implementations



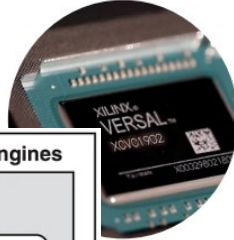
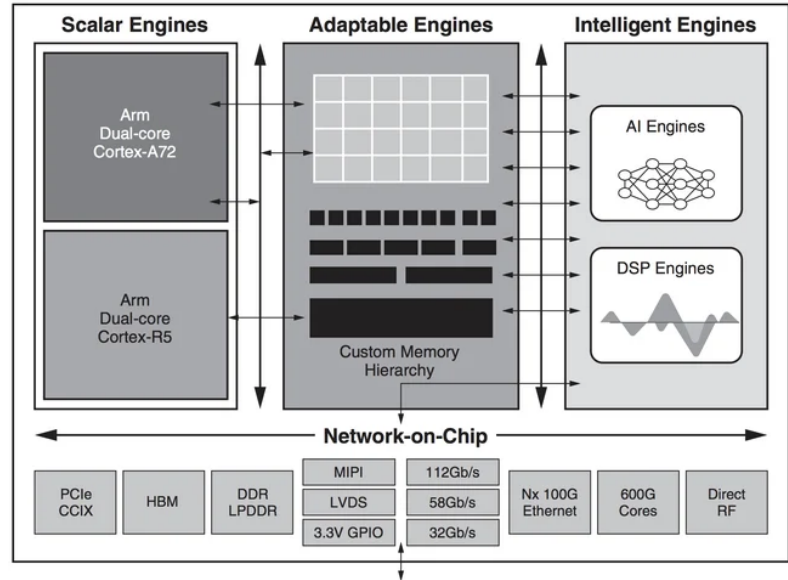
Modern FPGA architectures – Xilinx

Moreover, modern FPGA devices also integrate non-programmable specialized hard cores:
ARM processors, memory controllers, low latency and high bandwidth I/O, etc.

Zynq UltraScale+ MPSoCs

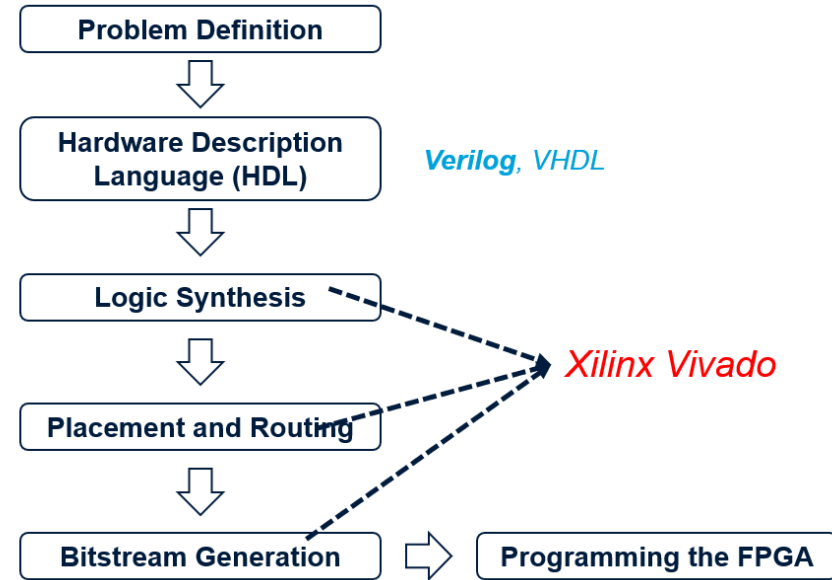


Versal Adaptive SoCs



Programming FPGAs

- Programming an FPGA means **configuring** the device by **loading configuration bits** that define how the internal logic resources (CLBs), interconnections, and memory blocks are arranged to implement a specific digital circuit
- The hardware architecture is translated into configuration bits (**bitstream**) used to program the device
- To design the hardware (obtain the bitstream), engineers typically use **Hardware Description Languages** (HDL), such as VHDL or Verilog
- HDLs are used to describe digital circuits at the **Register Transfer Level** (RTL), specifying how data moves between registers and how combinational logic operates between clock cycles

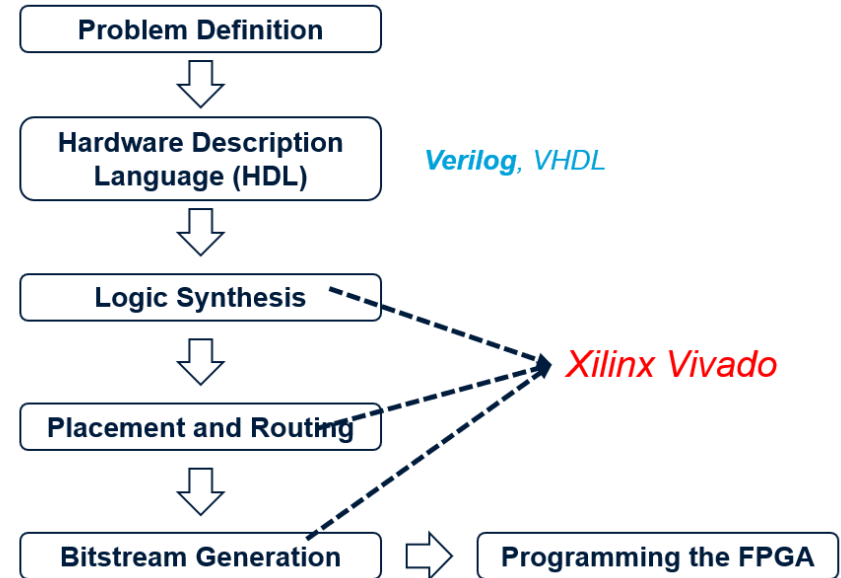


FPGA Design Flow

Programming FPGAs

- Once the HDL code is written, it is provided to FPGA design tools (logic synthesis, placement and routing, and bitstream generation): **the generated bitstream can then be loaded onto the FPGA to configure its internal SRAM cells!**

Actually, High-Level Synthesis (HLS) tools exist, which are compilers that allow to translate C or C++ codes in HDL codes.

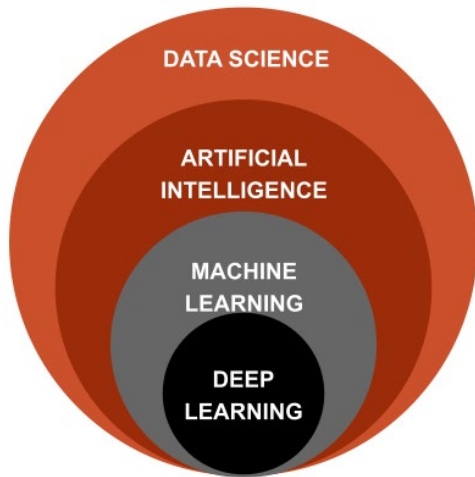


FPGA Design Flow

Neural Networks (NNs)

Understanding Artificial Intelligence (AI)

- AI is the field of computer science focused on developing algorithms, programs, and systems that can simulate, extend, or reproduce human cognitive abilities such as learning, reasoning, problem solving, pattern recognition, natural language processing, and adaptation to the environment.

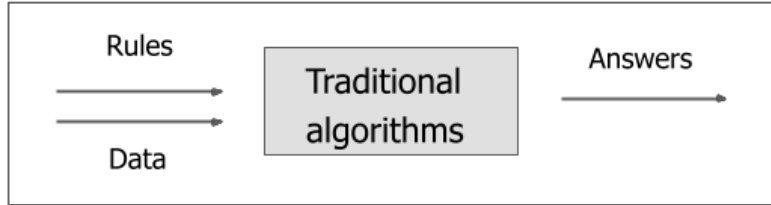


AI includes all computational methods that allow a machine to reason, make decisions, understand information, or act autonomously, whether they follow explicit rules (classic AI) or learn from data (ML).

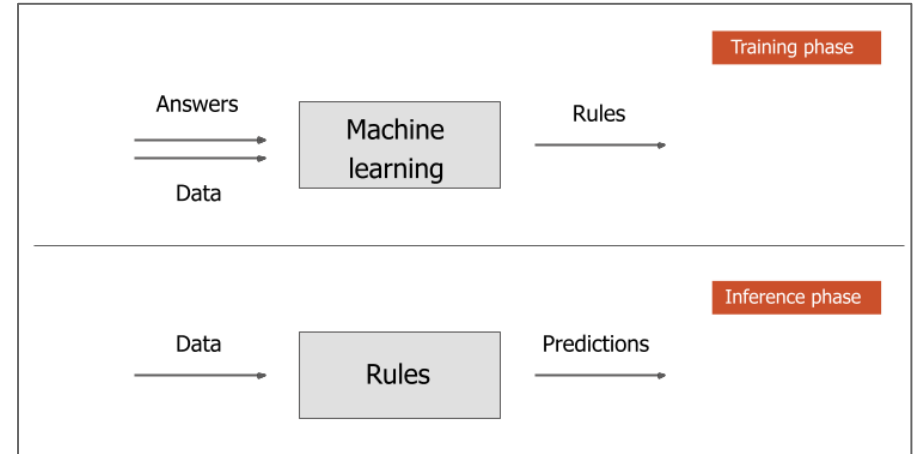
Machine Learning (ML) uses statistical models and optimization to automatically **extract patterns from data and learn rules over time** (training) that can make predictions and decisions

Deep Learning (DL) relies on deep neural networks (NNs) to learn complex patterns and high-level representations from large datasets. E.g.: CNNs, RNNs, Transformers, LLMs.

Traditional vs. ML/DL algorithms



- Humans define the rules (relations between data and answers) manually
- The computer uses these rules to compute the answer

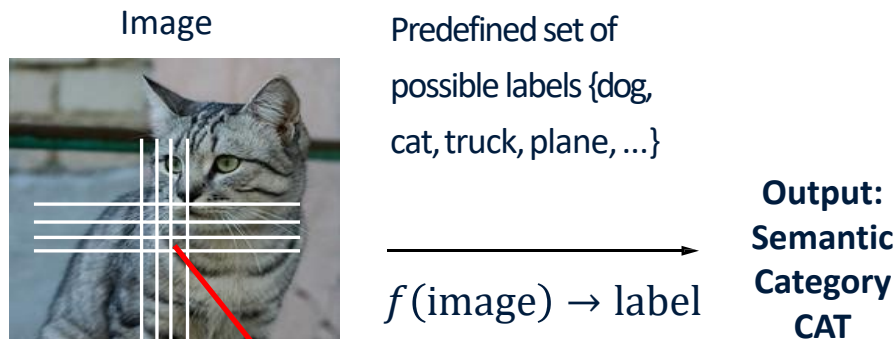


- Training phase: the model receives both data and the correct answers, and it learns the rules by itself by identifying **patterns** directly in the input data
- Inference phase: it applies the rules it learned in a set of new data

Example – Image Classification

Why do we need AI? Let's consider a core task in computer vision: Image classification.

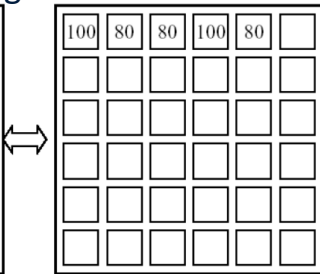
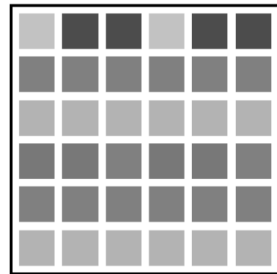
The problem: **Semantic Gap**



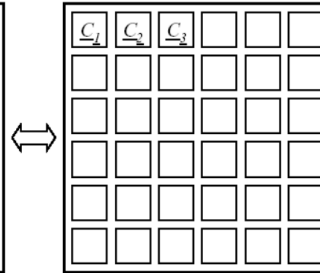
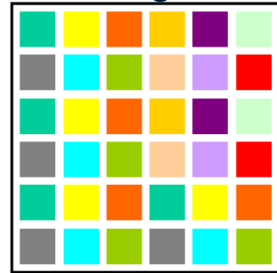
What the computer actually sees:
Pixel values $A(i, j)$

An image is a tensor of integers between $[0, 255]$:
e.g., $800 \times 600 \times 3$
(3 channels RGB)

monochrome image



color image

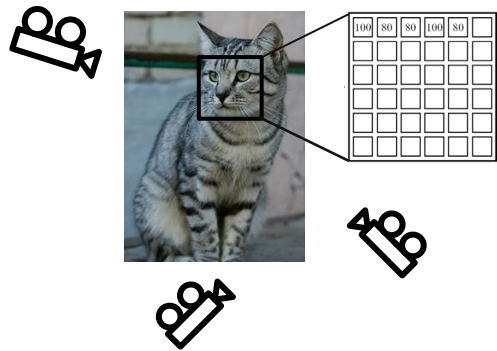


$$\underline{c}_i = [r_i \quad g_i \quad b_i]$$

Example – Image Classification

- The central challenge of computer vision is bridging the Semantic Gap: the difference between *low-level data* (pixel values) and *high-level meaning* (objects, concepts)
- This is not a trivial task for traditional algorithms which rely directly on pixel values or handcrafted rules because, for a given semantic meaning of an object, pixel values are highly sensitive to many factors!

Viewpoint Variation



Illumination



Deformation



Occlusion



Background clutter



Interclass variability



Example – Image Classification

What does it mean to “look like a cat”? The answer is complex and context-dependent

We cannot easily hard-code an image classifier:

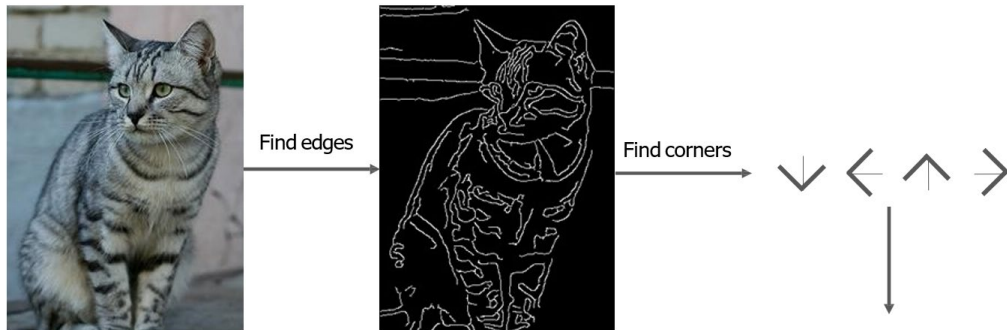
```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

For tasks like sorting numbers, we can write explicit algorithms. But for recognizing objects, there is no clear set of deterministic rules

Classical attempts have been made!

Before NNs, computer vision relied heavily on handcrafted pipelines. Typical steps included:

- | | |
|-----------------------|-------------------------------------|
| • Detect edges | Limitations: |
| • Detect corners | Manually designed features |
| • Extract descriptors | Performance sensitive to conditions |
| • Apply a classifier | Limited generalization |

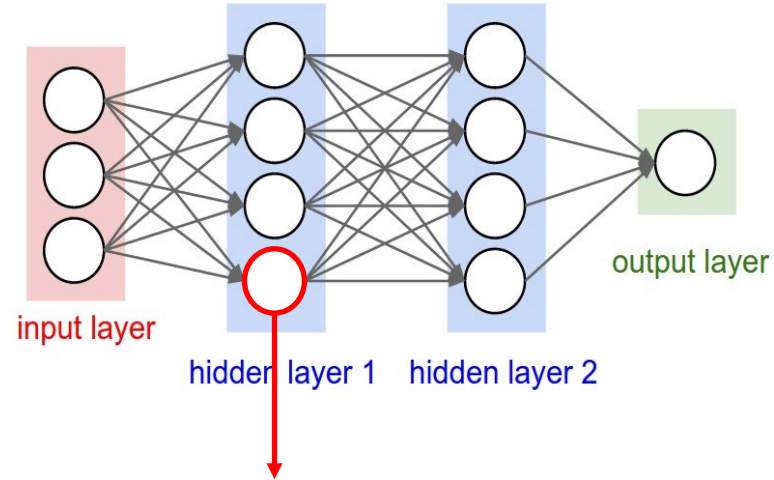


NNs address these challenges by learning representations/features rather than relying on raw pixels!

Neural Networks Architecture

- Deep learning uses *multilayered NNs*, which receive an input and transform it through a series of hidden layers to extract increasingly *abstract* and *meaningful representations* from the data. Each hidden layer is made up of a set of *artificial neurons*.

The architecture shown here is a *fully connected neural network*, where each neuron is "fully" connected to all neurons in the previous layer (receives the outputs of all neurons in the previous layer, forming dense connections between consecutive layers).

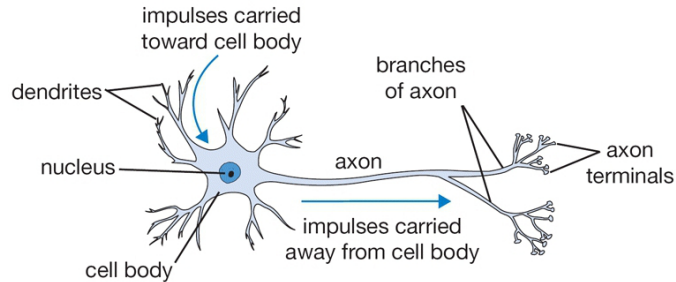


Each node (artificial neuron) is the fundamental computational unit of a NN.

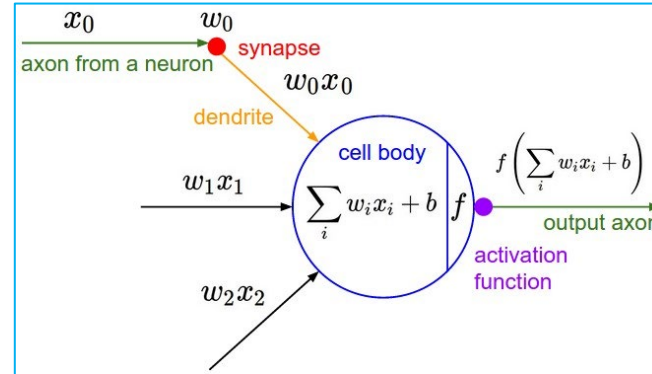
Artificial Neuron: Core Computation

Anatomical model: each neuron receives input signals from its dendrites and produces output signals along its axon

The axon is connected via synapses to the dendrites of other neurons



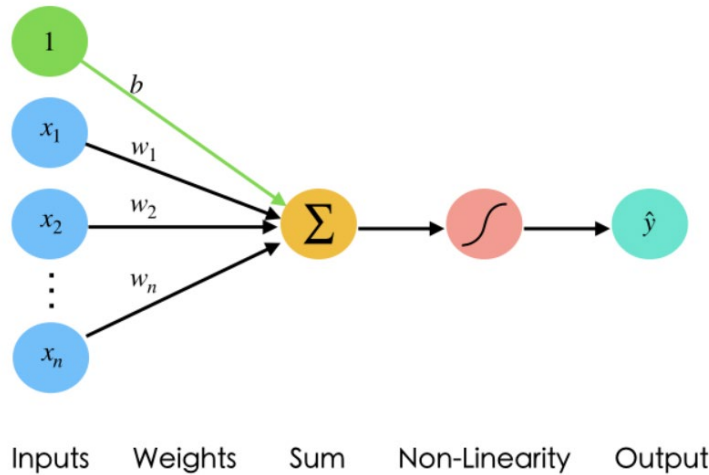
Computational model



- $\mathbf{x_i = inputs}$ (signal that travel along axons)
- $\omega_i = \mathbf{weights}$ (strength of the synapse, influence of one neuron on another)
- $\mathbf{b = bias}$ (offset, intrinsic excitability, activation threshold of the neuron)
- $\mathbf{f = activation}$ (non-linear function that determines how strongly the neuron fires, output response)

Artificial Neuron: Core Computation

- A neuron receives numerical inputs x_i from the previous layer of neurons, multiplies each input by a *weight* w_i , adds a *bias* term b , and applies an *activation function* f .
- The weights and biases are learned during training.**



linear combination of inputs

$$\hat{y} = f \left(b + \sum_{i=1}^m x_i w_i \right)$$

non linear activation function

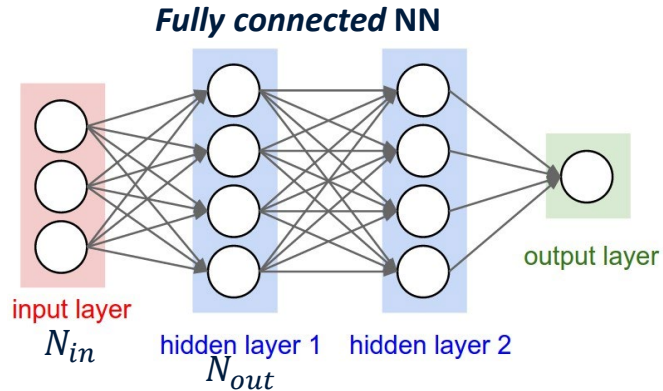
Weighted Sum: the core computation inside the neuron. Each input is multiplied by its weight and then accumulated. This operation corresponds to a **MAC (Multiply–Accumulate)**.

Vector/matrix multiplication

$$\hat{y} = f(b + \mathbf{X}^T \mathbf{W})$$

$$\mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Convolutional Neural Networks (CNNs)



Fully connected NN: Each neuron connects to all neurons in the previous and next layers.

Trainable parameters: every connection has a weight, and each neuron has a bias; each performs a MAC operation.

Two bottlenecks:

- Loss of spatial structure: images must be flattened into 1D vectors → spatial relationships between pixels are lost
- The number of parameters grows very quickly → Computationally expensive

For a fully connected (FC) layer, the number of trainable parameters is:

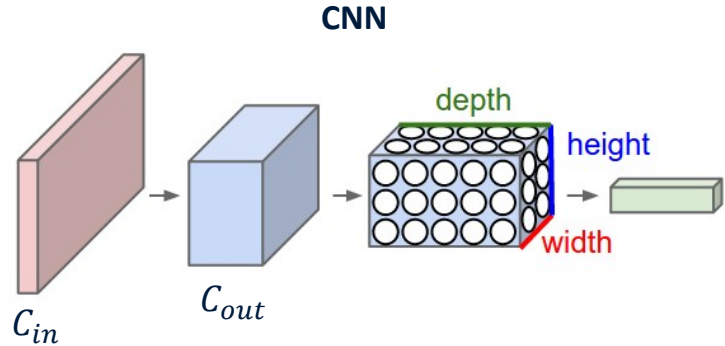
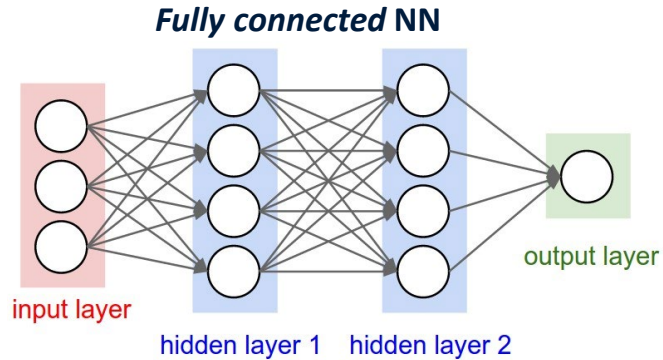
$$N_{params}^{FC} = N_{in} \times N_{out} + N_{out}$$

If the input is an image of size $H \times W \times C$ then:

$$N_{in} = H \times W \times C$$

The number of parameters grows with the spatial resolution of the image

Convolutional Neural Networks (CNNs)



To work effectively with images, a model must exploit:

- ✓ the **3D structure** of the image
- ✓ the **locality** of nearby pixels
- ✓ the **weight sharing** of local filters

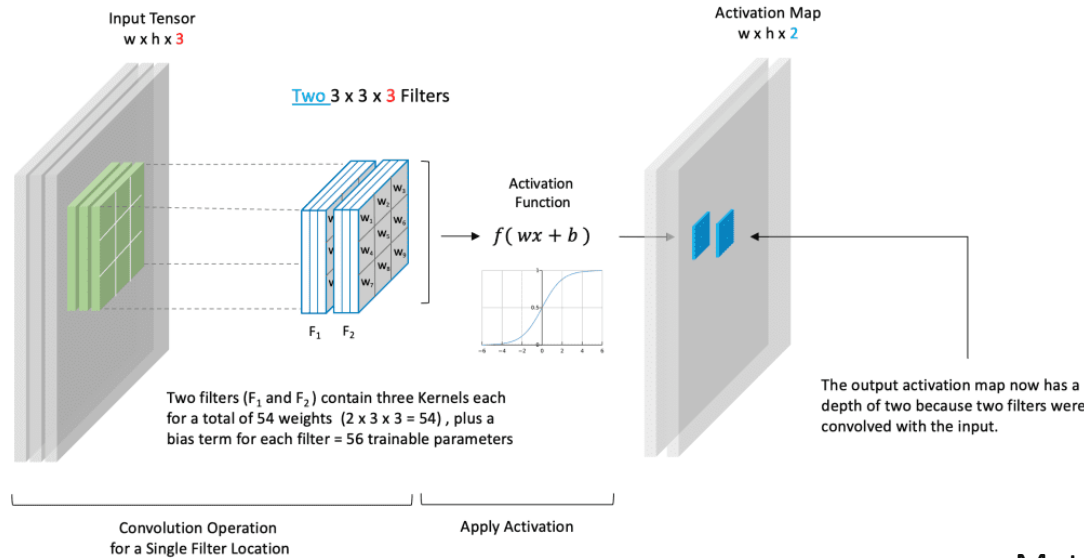
In a CNN, the image is not flattened but kept as a **volume** with width, height, and depth at every layer.

Depth (C_{out}) number of **channels** of the convolutional layer = number of feature maps

For a **convolutional (CNN) layer**: $N_{params}^{CNN} \propto C_{in} \times C_{out} + C_{out}$

C_{in} number of input channels (e.g., 3 for an RGB image)

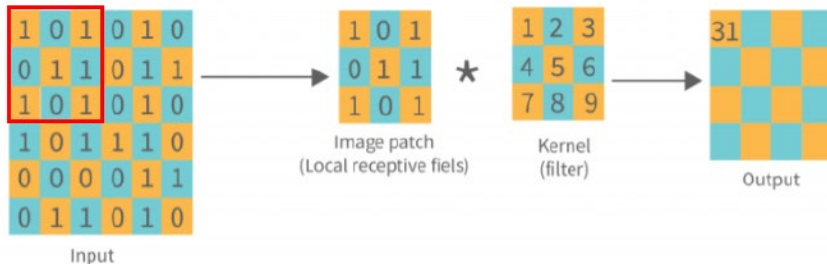
Convolutional Neural Networks (CNNs)



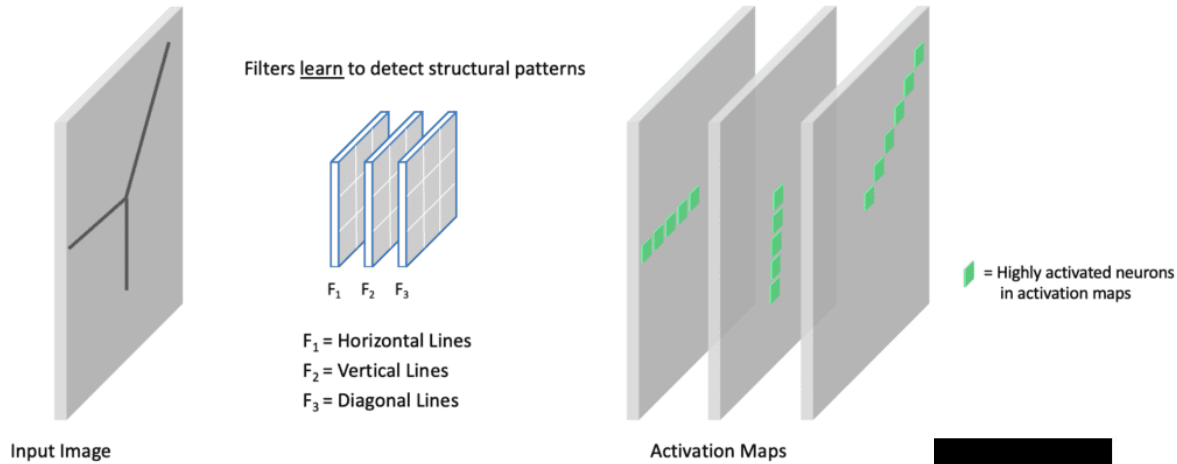
A convolutional layer uses a small **filter** (a matrix of trainable weights) to extract meaningful **features** from an image. The filter slides across the image and, at each position, it performs a **MAC operation** between its weights and the underlying pixels, producing a value in the output feature map

Matrix multiplication

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \cdot \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix} = \sum_{i=1}^3 \sum_{j=1}^3 a_{i,j} k_{i,j}$$

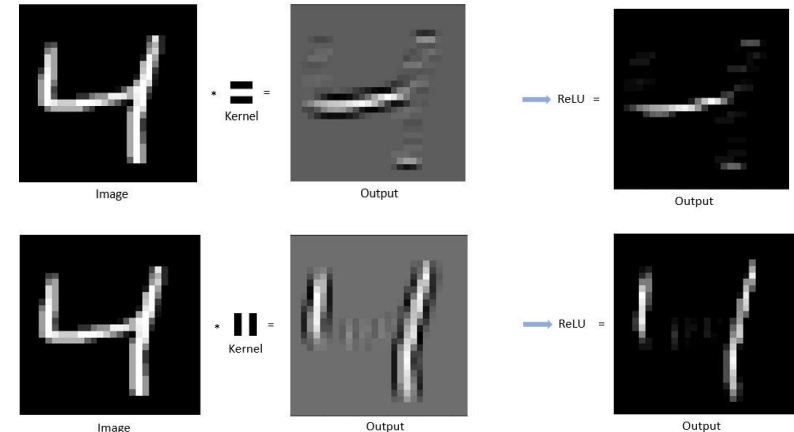


Convolutional Neural Networks (CNNs)

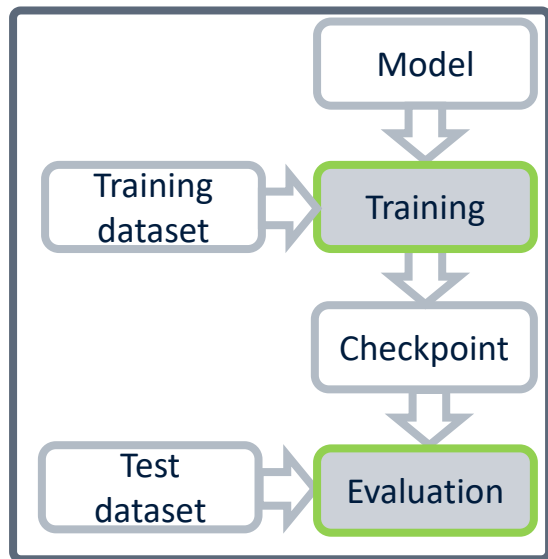


- Filters learn to detect specific structural patterns
- Filters isolate specific shapes or patterns of the input image, based on what they have learned to detect during the training process

The type of information extracted by each filter (such as edges, textures, or more complex patterns) depends entirely on the training process, during which the network learns the features that are most useful for the task



Training a Neural Network



Training stage

Feed Data: Expose the network to sample data

Compute Error: Compare predictions with labels

Update parameters: Adjust weights and biases

Repeat!: Iterate many times until convergence

Training consists of minimizing a loss function $\mathcal{L}$ with respect to the model parameters θ (weights and biases)

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i)$$

x_i = input sample

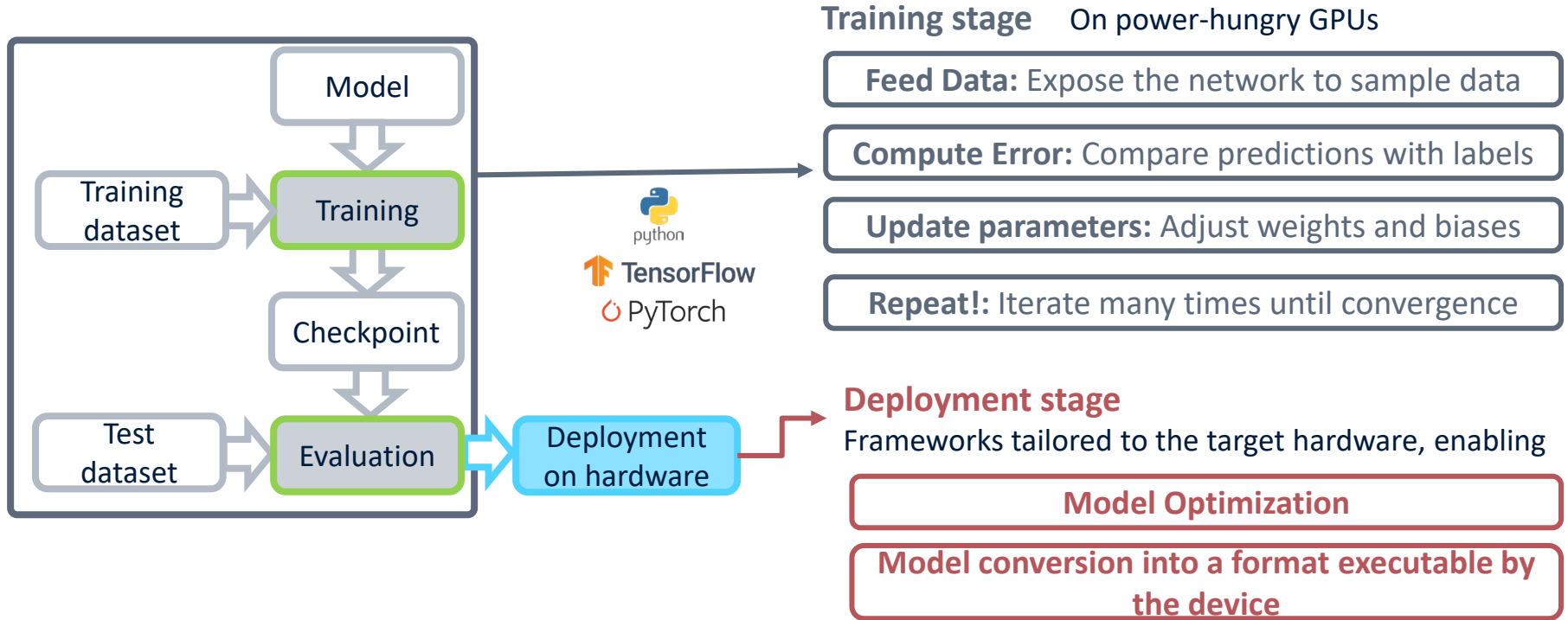
y_i = true label

$f_{\theta}(x_i)$ = model prediction

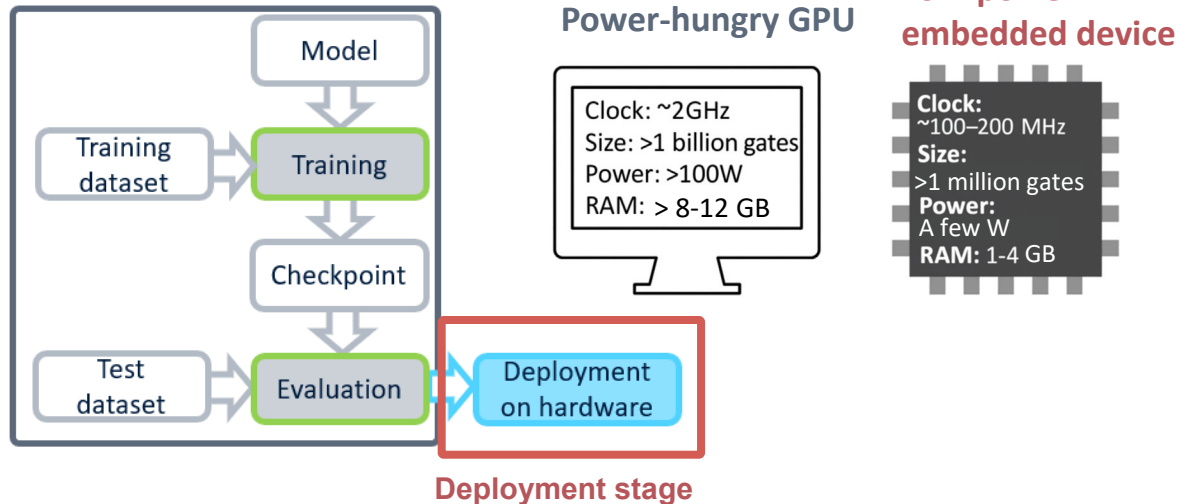
θ = weights and biases

$\ell(\cdot)$ = loss per sample

From Training to Deployment On Edge Devices



The Edge AI Challenges



Deployment Challenges

- **Limited Compute:** Edge devices lack high-performance CPUs/GPUs and have low clock speeds, making heavy DL models difficult to run in real time
- **Memory Constraints:** RAM and storage capacity are significantly reduced, restricting model size and intermediate RAM usage
- **Power Limitations:** Battery-powered or low-power embedded systems must balance performance with strict energy budgets

Two Key Ingredients for Edge AI Deployment

- Optimization techniques to reduce model complexity → TinyML approaches
- Specialized compute units → Hardware Accelerators

Takeaways

- Learning objectives of week 5
- What is an FPGA and what enables its programmability?
- What is a LUT and how can it be used to implement a combinational function?
- How to program an FPGA
- Neural Networks
- From training to deployment on edge devices: the edge AI challenges